RHEUMATRACK · TEAM KAIZEN

Step-by-step guide & project report

Multi-joint mobility monitoring and symptom-assessment web application

Covers: what was built and verified · running locally · MATLAB datasets and simulation · physician accounts · testing · connecting Firebase · pushing to GitHub from VS Code · deploying on Vercel · code walkthrough · final checklist.

Version: 1.0 · 26 October 2026

RheumaTrack is not a diagnostic system. Observed range of motion (ROM) is max − min angle within one recording and is not clinically validated. No sensor hardware is connected.

Contents

1. Project summary
2. What was built
3. Technology and architecture
4. Verification report
5. Final checklist
6. Run the project locally
7. MATLAB: datasets and simulation
8. Using simulated data in the app
9. Creating physician (clinician) accounts
10. Testing that everything works
11. Connect to a real Firebase project
12. Push to GitHub from VS Code
13. Deploy on Vercel
14. Troubleshooting
15. Code walkthrough
16. Appendix A — file index
17. Appendix B — command reference

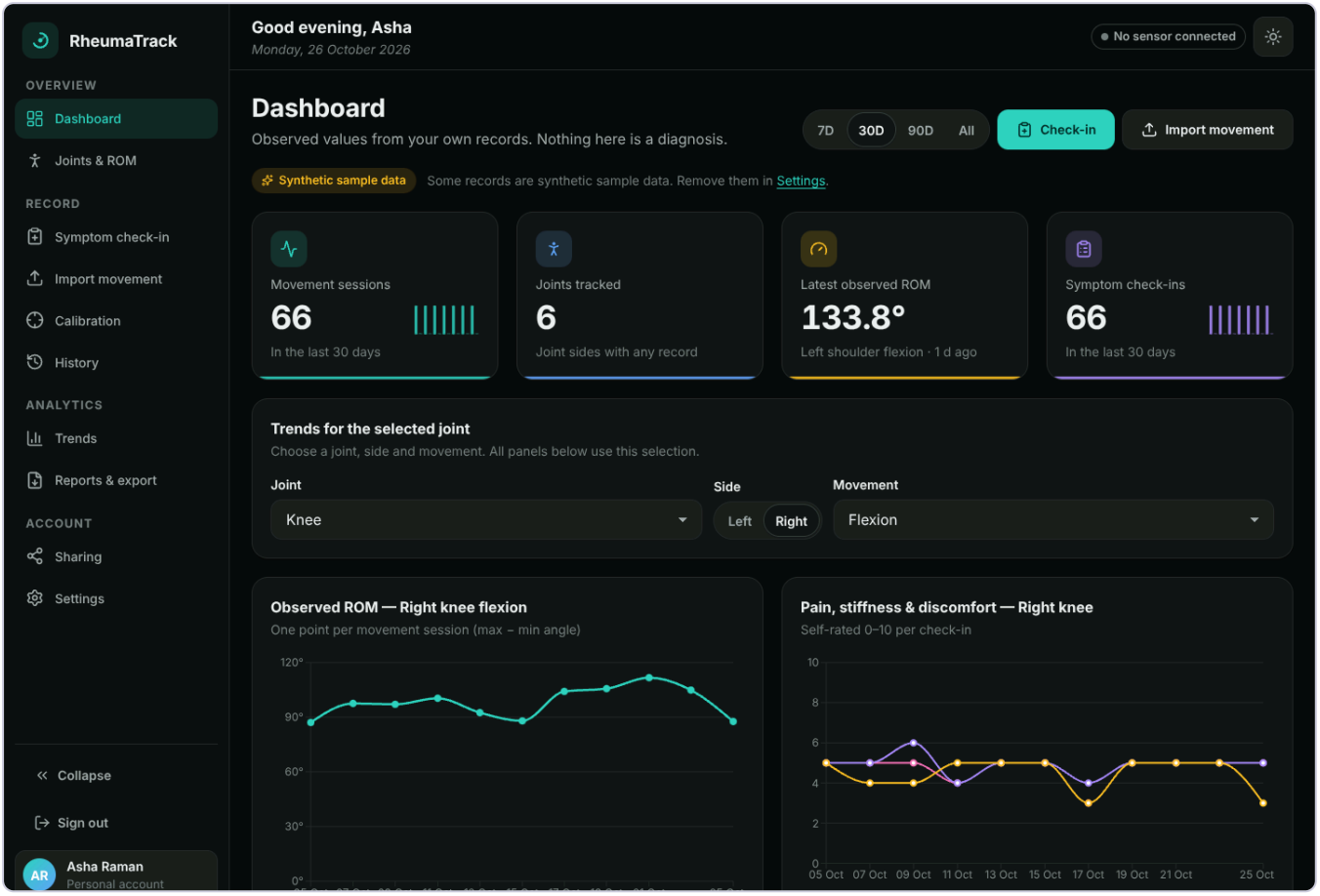
Where is the code? The complete source code is in `rheumatrack.zip` (≈150 source files, ≈11 500 lines of TypeScript plus MATLAB, Python, rules and tests). This guide explains it section by section with excerpts taken directly from the files; Appendix A lists every folder and key file.

1. Project summary

RheumaTrack is a web application in which a person can:

- record pain, stiffness and discomfort for 12 joints (left/right shoulder, elbow, wrist, hip, knee, ankle);
- import timestamped joint-angle recordings (CSV/JSON) generated in MATLAB or acquired elsewhere;
- see observed range of motion per session, trends over time, left/right comparisons and a statistically qualified symptom ↔ ROM association;
- explore an interactive 3D body: clicking a joint zooms to it, animates its movement and shows its observed range;
- export CSV/JSON and print clinician reports;
- share read-only access with a physician they explicitly invite, and revoke it at any time.

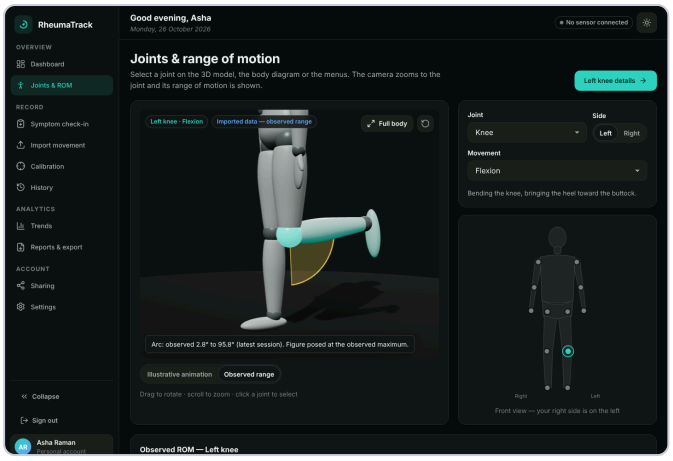
The backend is Firebase (Authentication + Cloud Firestore with Security Rules). It runs fully locally on the Firebase Emulator Suite, and can be connected to a real Firebase project and deployed to Vercel or Firebase Hosting.



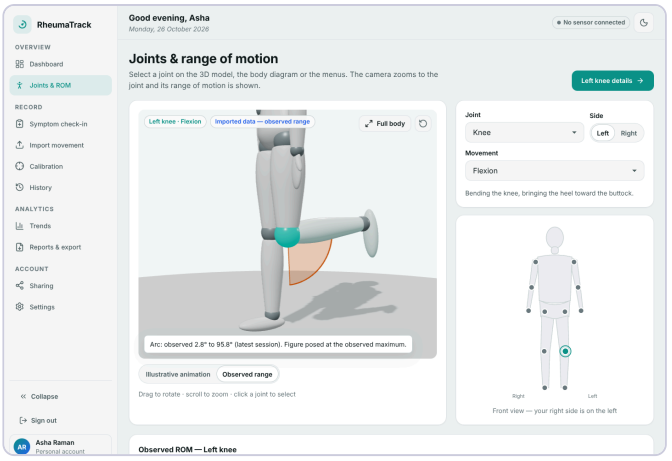
Dashboard (dark theme) with synthetic sample data: stat cards, ROM and symptom trends.

2. What was built

Area	Delivered
Authentication	Sign-up, sign-in, sign-out, password reset, persistent sessions, protected routes, clear error messages, email verification.
Roles & consent	Patient and clinician roles; clinician role stored server-side (`/roles`, Admin/console only). Invite → accept → share → revoke flow, enforced by Security Rules.
UI / UX	Charcoal + teal dark theme and full light theme; one-click toggle, OS preference on first visit, no flash on load; collapsible sidebar with tooltips (remembered); mobile drawer; skeletons, empty states, toasts, confirmation dialogs; 44 px touch targets; keyboard accessible.
Joints	12 joint sides; synchronised selection from the 3D model, body diagram, joint dropdown and side toggle; joint detail pages with movements, 3D view, ROM rings, session and symptom history, calibration status and exports.
3D viewer	Procedural articulated mannequin (three.js / React Three Fiber); clickable joints; camera zoom per joint; illustrative animation with play/pause/replay/timeline and angle readout; observed-range arc; separate imported-sample playback on session pages.
Movement analysis	CSV/JSON import with row-level validation (columns, timestamps, units, joint/movement, duplicates, ordering, gaps, sampling rate, size); observed min/max/ROM, mean, duration, interval statistics, quality indicators, hysteresis repetition count.
MATLAB	Synthetic generator, analyser, CSV/JSON exporter for your own data, trend dataset, golden verification against the Python reference and the web app.
Calibration	Joint-specific static-reference workflow (instructions, source, validation, retry/cancel, metadata). Never claims sensor calibration.
Symptoms	0–10 sliders for pain/stiffness/discomfort, duration, notes, timestamp; edit/delete; history never overwritten.
Analytics	Dashboard cards from real records only, ROM/symptom trends, left/right comparison, joint comparison, Spearman association with ≥ 10 pairs and stated limitations.
Reports	Sessions/check-ins CSV, JSON bundle, raw-sample export, printable clinician and session reports — all labelled with data source and synthetic status.
Reminders	Schedule with timezone, in-app due banner, downloadable .ics calendar reminder (no push — stated in UI).
Data control	Delete individual records, remove all synthetic data, delete account and all data.
Testing	135 unit/integration tests, 18 Security Rules tests, 9 browser E2E tests, golden cross-implementation checks.
Docs	README, architecture, data contract, security, Firebase setup, Vercel, analytics, calibration, 3D viewer, IMU interface, verification report, remaining work.



Joint selected → camera zooms in; arc = observed range from imported data.



Same view in the light theme.

3. Technology and architecture

Layer	Technology (installed version)
Frontend	React 19.3, TypeScript 5.9 (strict), Vite 8.3, React Router 7.18, CSS design tokens
3D	three 0.186, @react-three/fiber 9.8, @react-three/drei 10.7 (no downloaded models)
Charts / icons / CSV	Recharts 3.10, lucide-react, PapaParse 5.7
Backend	Firebase JS SDK 12.19: Authentication (email/password), Cloud Firestore, App Check (optional); firebase-tools 15 (emulators, deploy); firebase-admin 14 (scripts)
Offline analysis	MATLAB scripts; Python 3 reference implementation (standard library only)
Testing	Vitest 5 + Testing Library (jsdom), @firebase/rules-unit-testing 5, Playwright 1.64
Hosting	Vercel (`vercel.json`) or Firebase Hosting (`firebase.json`)

How the pieces connect

1. **Pages** (`src/features/*`) render the UI and call **providers** (`src/app/providers`) for the signed-in user, their role, their records, the selected joint and the theme.
2. Pages never call Firebase directly; they use the **service interfaces** in `src/data/services.ts`. The Firestore implementation is in `src/data/firestore`; an in-memory implementation is used only by automated tests.
3. All calculations live in **pure functions** in `src/domain` (validation, ROM, repetitions, correlation, exports, reminders). The same definitions exist in MATLAB and Python, and tests prove they give identical results.
4. **Firestore Security Rules** (`firebase/firestore.rules`) enforce who can read and write what — the UI is never trusted for permissions.

Data model (Cloud Firestore)

Path	Contents

`roles/{uid}`	`role: "clinician"` — written only from the console or Admin SDK
`users/{uid}`	profile: name, email, reminder settings
`users/{uid}/sessions/{id}`	joint, side, movement, data source, synthetic flag, timestamps, metrics, calibration link, notes
`users/{uid}/sessions/{id}/chunks/{n}`	raw samples, up to 5000 per document (`t` = epoch ms, `a` = degrees)
`users/{uid}/assessments/{id}`	pain, stiffness, discomfort, duration, notes, assessment time
`users/{uid}/calibrations/{id}`	static-reference calibration results (immutable)
`invites/{id}`, `shares/{patient}_{clinician}`	consent-based sharing

4. Verification report

Result	Check	Details
PASSED	TypeScript strict check (`npm run typecheck`)	0 errors
PASSED	Unit + integration tests (`npm test`)	135 / 135 passing (repeated runs)
PASSED	Golden cross-check TypeScript vs Python	4 datasets equal within 1e-9
PASSED	Production build (`npm run build`)	succeeds, no warnings
PASSED	Browser E2E against in-memory build	8 / 8 workflow tests (all 10 required workflows) in real Chromium with WebGL
PASSED	No console errors; no horizontal overflow at 1440 px and 390 px	checked by script
PASSED	Clean install from the ZIP	install, typecheck, tests and build all pass in an empty folder
PASSED	Your PC: `npm install`; Vite dev server	installed (954 packages); app loads at :5173
NOT RUN	Firebase emulators on your PC	stopped: Java 17 found, firebase-tools 15 needs Java 21 — install Java 21 (section 6)
NOT RUN	Security Rules tests (`npm run test:rules`)	18 tests written; need Java 21 + emulator (your PC)
NOT RUN	E2E against emulators (`npm run test:e2e`)	9 tests written; your PC
NOT RUN	MATLAB scripts	not available in the build environment; run in MATLAB Online (section 7)
NOT RUN	Firebase project / Vercel deployment	not created yet — sections 11–13

Issues found and fixed during development: test selector issues, URL ↔ joint-selection sync, sign-out hidden in the mobile drawer, date/time field truncation, a cold-start test timeout, and the documented Java requirement (corrected from 11+ to 21+). Known advisory: `npm audit` reports issues in `@grpc/grpc-js`, which only

Firebase's Node.js tooling uses (not the browser app). Do not run `npm audit fix --force`; it downgrades Firebase.

5. Final checklist

Delivered and verified

✔	Complete source code, clean project, ZIP
✔	Authentication, protected routes, roles, consent sharing
✔	Light/dark themes, collapsible sidebar, responsive layout
✔	Body diagram + 3D viewer with joint zoom and ROM
✔	CSV/JSON validation and ROM analysis with tests
✔	MATLAB scripts + synthetic and invalid sample datasets
✔	Symptoms, history, analytics, association, reports, exports
✔	Calibration workflow, reminders, data deletion
✔	Firestore rules, indexes, .env.example, firebase.json, vercel.json
✔	Automated tests + verification report + documentation

Your actions (in order)

☐	Install Java 21 and start `npm run dev:local` (section 6)
☐	Run `npm run seed`, sign in, try every page (section 10 manual checklist)
☐	Run `npm test`, `npm run test:rules`, `npm run test:e2e` (section 10)
☐	Run the MATLAB scripts in MATLAB Online and import the files (sections 7–8)
☐	Create the Firebase project, fill `.env.local`, deploy rules (section 11)
☐	Create a physician account on the real project (section 9)
☐	Push to GitHub from VS Code (section 12)
☐	Import into Vercel, add env vars + authorised domain, test the live site (section 13)
☐	Before real patients: privacy/legal review and items in `docs/REMAINING_WORK.md`

6. Run the project locally

Use the VS Code terminal (**Terminal → New Terminal**) in `C:\dev\rheumatrack`. Keep the project outside OneDrive.

6.1 One-time prerequisites

Tool	Install (Windows)	Check
Node.js 22 or 24 LTS	`winget install OpenJS.NodeJS.LTS`	`node -v`
Java 21 (required by the emulators)	`winget install EclipseAdoptium.Temurin.21.JDK`	`java -version` must say 21
Git	`winget install Git.Git`	`git --version`

After installing anything, close and reopen VS Code (or refresh the PATH in the open terminal):

```
$env:Path = [Environment]::GetEnvironmentVariable("Path","Machine") + ";" +
[Environment]::GetEnvironmentVariable("Path","User")
```

If `java -version` still shows 17, point this terminal at Java 21 (or remove Java 17 / reorder it in System → Environment Variables → Path):

```
$jdk = (Get-ChildItem "C:\Program Files\Eclipse Adoptium" -Directory | Where-Object Name -like
"jdk-21*" | Select-Object -First 1).FullName
$env:JAVA_HOME = $jdk
$env:Path = "$jdk\bin;" + $env:Path
java -version
```

If PowerShell says “running scripts is disabled”, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

6.2 Install and start

```
npm install            # first time only (done on your PC)
npm run dev:local      # emulators + app; wait for "All emulators ready!" and "Local:
http://localhost:5173/"
```

In a **second terminal** (the **+** button):

```
npm run seed
```

Open **http://localhost:5173**. Accounts created by the seed (local only, synthetic data):

Role	Email	Password
Patient	patient@rheumatrack.test	Patient123
Physician (clinician)	clinician@rheumatrack.test	Clinician123

> The Emulator UI at **http://127.0.0.1:4000** shows users (Authentication tab) and all documents (Firestore tab). Emulator data is erased when you stop it with Ctrl+C; use `npm run emulators:persist` + `npm run dev:emu` (two terminals) to keep data between runs.

7. MATLAB: datasets and simulation

> There is no real patient dataset in this project, and none should be invented. All provided data is **synthetic** (simulation-generated) and labelled as such. To use real measurements later, convert them with `rt_export_recording.m` (7.5).

7.1 Datasets you already have

Folder / file	What it is	Use it to
`matlab/samples/synthetic_*.csv/.json`	8 synthetic recordings (shoulder, elbow, wrist, hip, knee ×2, ankle)	import and see ROM, repetitions, 3D playback
`matlab/samples/trend/…day1–day7.csv`	7 daily right-knee sessions with gradually increasing range	see the ROM trend chart
`matlab/samples/synthetic_static_hold_knee_right.csv`	3-second still hold	complete the calibration workflow
`matlab/samples/invalid_*.csv`	4 deliberately broken files	see row-level rejection messages
`reference/golden/*.csv` + `expected.json`	fixed inputs + reference results	prove MATLAB = Python = web app

7.2 The simulation model

Each synthetic recording is a smooth repeated movement plus random noise:

```
angle(t) = baseDeg + amplitudeDeg · (1 − cos(2π · t / periodS)) / 2 + noise,   noise ~ N(0, noiseSdDeg²)
```

You choose joint, side, movement, sampling rate, duration, start angle, amplitude, seconds per repetition, noise and a random seed. The core of `rt_generate_synthetic.m`:

```
idx = (0:n-1)';
tS = idx * intervalMs / 1000;
rng(opts.seed, 'twister');
clean = opts.baseDeg + opts.amplitudeDeg * (1 - cos(2*pi*tS/opts.periodS)) / 2;
noisy = clean + opts.noiseSdDeg * randn(n, 1);
angleDeg = round(min(max(noisy, cfg.minAngleDeg), cfg.maxAngleDeg) * 100) / 100;
```

7.3 Run it in MATLAB Online (step by step)

1. Go to **https://matlab.mathworks.com** and sign in (a free MathWorks account works; universities often provide a licence).
2. In the **Files** panel create a folder `rheumatrack`. Upload (drag and drop) the project's `matlab` and `reference` folders into it — keep them side by side.
3. In the Command Window, go to the scripts folder:

```
cd rheumatrack/matlab
```

Generate one recording (writes `.csv` and `.json`):

```
opts = struct('joint','knee','side','left','movement','flexion', ...
              'sampleRateHz',50,'durationS',20,'amplitudeDeg',115,'periodS',4, ...
              'noiseSdDeg',1.2,'seed',7,'outBase','knee_left_flexion_synthetic');
[t, a] = rt_generate_synthetic(opts);
```

Analyse it (same definitions as the web app):

```
m = rt_analyse_session(rt_read_movement_csv('knee_left_flexion_synthetic.csv'))
```

Output fields: `sampleCount, minDeg, maxDeg, romDeg, meanDeg, durationMs, interval.*, irregularIntervals, repetitions`. Plot it if you like:

```
plot((t - t(1))/1000, a); xlabel('time (s)'); ylabel('angle (deg)'); grid on
```

Regenerate all sample sets and **the 7-day trend set**:

```
rt_make_sample_datasets   % 8 recordings into samples/
rt_make_trend_dataset     % 7 daily knee sessions into samples/trend/
```

Download the files: right-click in the Files panel → **Download**.

7.4 Allowed values

Joint	Movements
shoulder	flexion, extension, abduction
elbow, wrist, hip, knee	flexion, extension
ankle	dorsiflexion, plantarflexion

Sampling rate 1–1000 Hz; 10–50 000 samples; angles −180…180°; side `left` / `right`.

7.5 Converting your own recordings

If you have joint angles from a goniometer, phone app or IMU (already converted to degrees), export them in RheumaTrack format:

```
t0  = datetime('2026-10-26 09:00:00','TimeZone','Asia/Kolkata');
tMs = round(posixtime(t0)*1000) + (0:999)'*20;     % 50 Hz timestamps
angle = myAngles(:);                               % your angles in degrees
rt_export_recording(tMs, angle, 'knee', 'left', 'flexion', 'my_knee_session', 'imported');
```

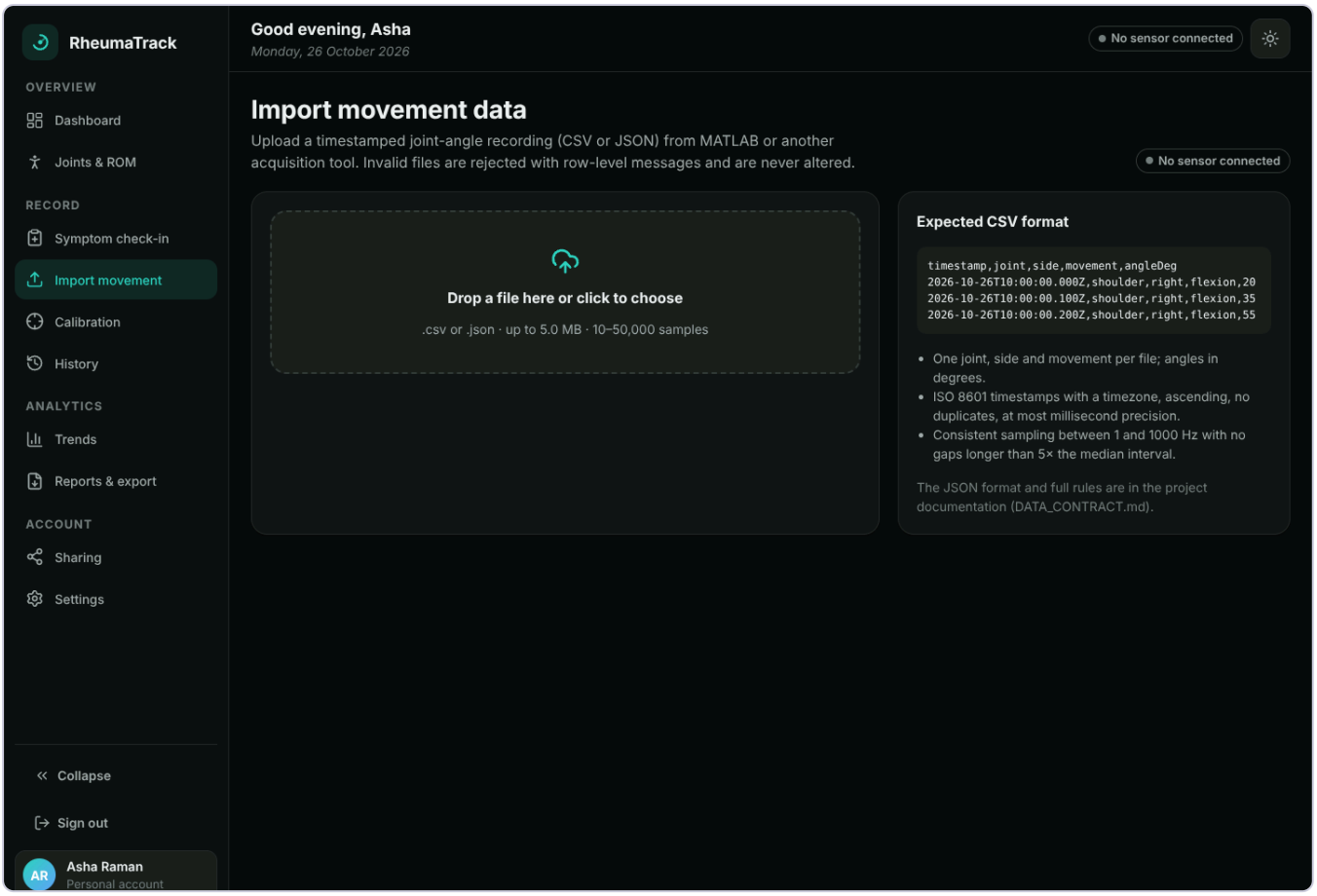
7.6 Prove MATLAB and the web app agree

```
rt_verify_golden
```

Expected: four lines ending in **PASS** and “All 4 golden files match the reference within 1e-09.” The web app runs the same comparison in `npm test` (`golden.test.ts`).

```
m.sampleCount = numel(angleDeg);
m.minDeg = min(angleDeg);
m.maxDeg = max(angleDeg);
m.romDeg = m.maxDeg - m.minDeg;
m.meanDeg = mean(angleDeg);
m.durationMs = tMs(end) - tMs(1);
m.interval.min = min(d);
m.interval.max = max(d);
m.interval.mean = mean(d);
m.interval.median = med;
m.interval.sd = std(d, 1);          % population SD, matches the app
m.interval.rateHz = 1000 / med;
m.irregularIntervals = sum(abs(d - med) > cfg.irregularTolerance * med);
m.repetitions = rt_count_repetitions(angleDeg, cfg);
```

8. Using simulated data in the app



Import movement page.

8.1 Import a recording

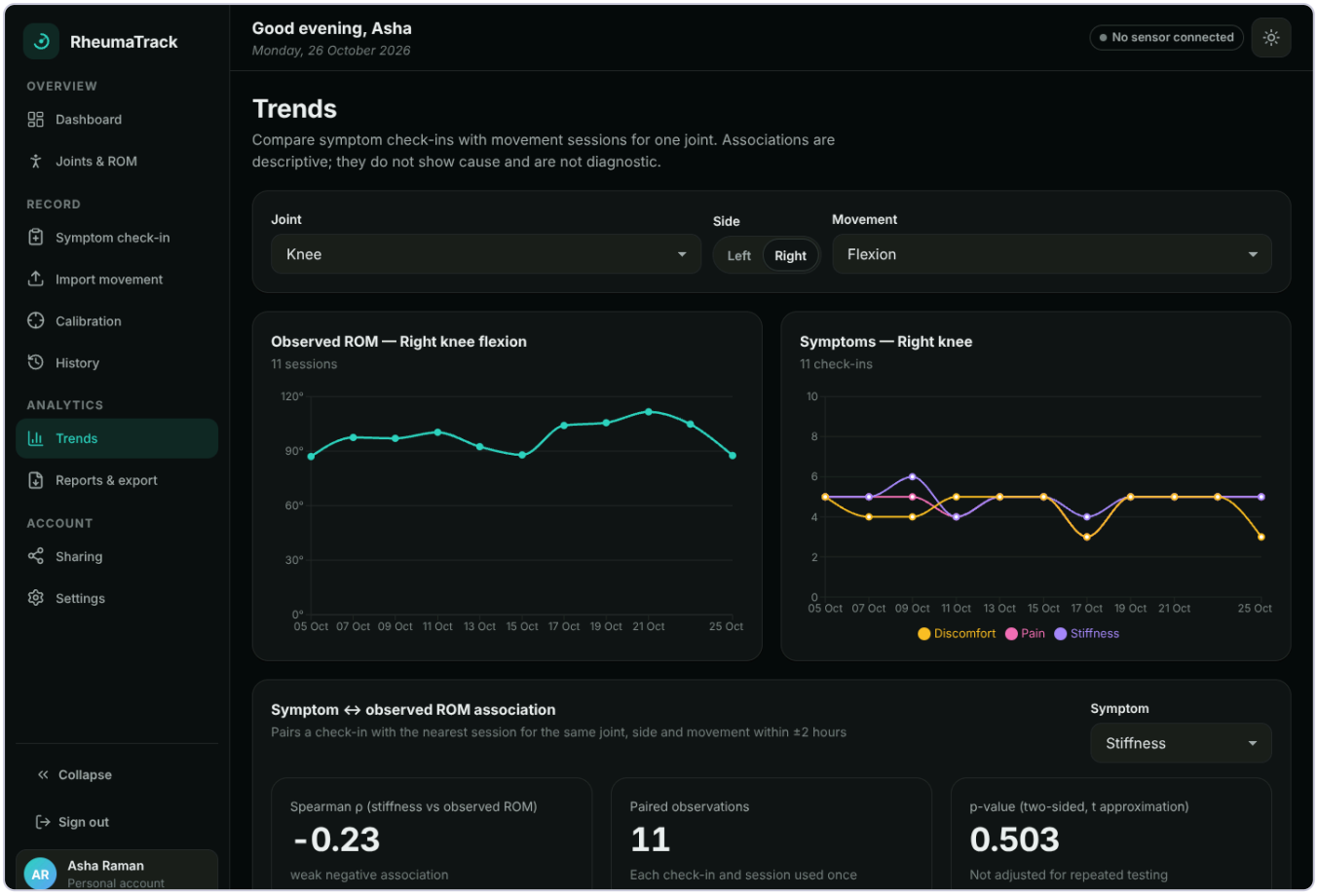
1. Sidebar → **Import movement** → drop a `.csv` / `.json` file.
2. Invalid files show a table of problems (row, field, message) and nothing is saved. Try `invalid_values.csv`.
3. Valid files show the angle chart, observed ROM, min, max, mean, duration, sampling rate and repetitions.
4. Choose **Where did this recording come from?** → *Simulation-generated* for MATLAB synthetic data (it will be labelled “Synthetic sample data”), or *External recording* for real data. Optionally link a static reference. Click **Save session**.
5. The session page plays the recording on the 3D figure, synced to the chart (play, pause, step, replay, scrubber). **How is this calculated?** explains each value.

8.2 See a trend

1. Import `trend/…day1.csv` to `day7.csv` one by one (Simulation-generated).
2. Dashboard → choose **Knee**, **Right**, **Flexion**, range **All** → the ROM trend shows 7 points.
3. For an association on **Trends**: record a right-knee check-in for each session, setting the check-in time within 2 hours of that session's start (09:00 IST on each day). An association value appears only after **10 or more** pairs — add more sessions (generate more days in MATLAB) to reach 10.

8.3 Calibration

1. Sidebar → **Calibration** → select Knee / Right / Flexion → **Start calibration**.
2. Choose `synthetic_static_hold_knee_right.csv` → all checks pass → **Save static reference**.
3. When importing a right-knee flexion session, pick it under **Static reference**. Angles are not changed; the link is metadata.



Trends page: ROM and symptom trends, Spearman association with paired observations and method notes.

9. Creating physician (clinician) accounts

> Everyone who signs up is a **patient**. A physician is a normal account that an administrator promotes by writing `roles/{uid} = { role: "clinician" }`. The app itself can never do this — that is what keeps the role trustworthy. A physician sees a patient only after that patient invites them and they accept.

9.1 Locally (emulator) — three options

A. Seeded account: `npm run seed` already creates `clinician@rheumatrack.test` and shares the demo patient with it.

B. Script (for any account you signed up locally):

```
npm run grant-clinician -- --emulator doctor@example.com
```

C. Emulator UI: open http://127.0.0.1:4000 → **Authentication** → copy the user's UID → **Firestore** → **Start collection** `roles` → Document ID = UID → field `role` (string) = `clinician`.

The physician signs out and in. In the emulator, email verification links appear in the terminal running the emulators.

9.2 On the real Firebase project

1. The physician signs up on the live site, opens **Settings → Verify email** and clicks the link in the email.
2. Firebase console → **Authentication → Users** → copy the physician's **User UID**.
3. **Firestore Database → Data → + Start collection** → Collection ID `roles` → Document ID = the UID → field `role`, type *string*, value `clinician` → **Save**.
4. The physician signs out and back in — the sidebar shows **Patients**.
5. Revoke: delete that `roles` document (and the physician's `shares` documents).

Alternative (scriptable): `npm run grant-clinician -- --project <project-id> doctor@example.com` with `GOOGLE_APPLICATION_CREDENTIALS` pointing to a service-account key stored **outside** the project folder.

9.3 Connecting a patient and a physician

1. Patient: **Sharing** → physician's email → tick consent → **Send invitation** (status “Waiting for clinician”). The app does not send emails — tell the physician.
2. Physician: **Patients → Pending invitations → Accept**. The patient appears under “Shared with you”; their pages open read-only.
3. Patient can **Revoke access** at any time; access ends immediately.

The rule that enforces this:

```
    match /roles/{uid} {
      allow read: if isOwner(uid);
      allow write: if false;
    }
...
    function hasShare(patientUid) {
      return signedIn()
        && exists(/databases/$(database)/documents/shares/$(patientUid + '_' +
request.auth.uid))
        && isClinician();
    }
    function canRead(uid) {
      return isOwner(uid) || hasShare(uid);
    }
```

10. Testing that everything works

10.1 Automated tests

```
npm run typecheck          # TypeScript: expect no output / exit 0
npm test                   # expect "Tests 135 passed (135)"
npm run build              # expect "built in ..."
npm run test:rules         # needs Java 21: expect 18 passed
npm run test:e2e:install   # downloads Playwright's Chromium (once)
npm run test:e2e:harness   # in-memory browser tests: expect 8 passed, 1 skipped
# full-stack browser tests: terminal 1 "npm run emulators", terminal 2:
npm run seed
npm run test:e2e           # expect 9 passed
```

On failure Playwright writes screenshots and traces to `test-results/`; open the report with `npx playwright show-report`.

10.2 Manual checklist (about 20 minutes)

☐	Area	Expected result
☐	**Sign-in**	Wrong password shows an error; correct password opens the Dashboard; Sign out returns to login.
☐	**Sign-up**	Create a new account; weak password / mismatched confirmation are rejected; new account starts with empty states (no invented numbers).
☐	**Password reset**	Forgot password → link appears in the emulator terminal (real project: email).
☐	**Theme**	Sun/moon button switches every page, chart and the 3D viewer; reload keeps the choice.
☐	**Sidebar**	Collapse → icons with tooltips; reload keeps it; mobile width → menu drawer.
☐	**Joints**	Click a joint on the 3D model and on the body diagram; change joint/side dropdowns — all stay in sync and the camera zooms; Full body / Esc zooms out.
☐	**Import**	Valid file → metrics → save; invalid files → row errors, nothing saved.
☐	**Session**	Playback moves the figure and the chart cursor; CSV/JSON export; report prints.
☐	**Check-in**	Invalid duration rejected; saved check-in appears in History; edit and delete with confirmation.
☐	**History**	Filters by joint, side, movement, dates; delete asks for confirmation.
☐	**Trends**	Fewer than 10 pairs → explanation instead of a number.
☐	**Reports**	CSV/JSON downloads open in Excel/editor; values match the session page.
☐	**Calibration**	Static hold passes; moving file fails with reasons; retry/cancel work.
☐	**Sharing**	Invite → physician accepts → read-only view → patient revokes → physician loses access.
☐	**Settings**	Reminder time/days save; .ics opens in Calendar; Remove synthetic data; Delete account.

11. Connect to a real Firebase project

1. Go to **https://console.firebase.google.com** → **Create a project** (e.g. `rheumatrack-prod`). Google Analytics is not needed. The free **Spark** plan is enough.
2. **Build → Authentication → Get started → Sign-in method → Email/Password → Enable → Save.** Under **Settings**: keep *Email enumeration protection* on; set a password policy (≥ 8 characters).
3. **Build → Firestore Database → Create database** → choose a location near your users (e.g. `asia-south1` Mumbai; cannot be changed later) → start in **production mode**.
4. **Project settings (gear) → General → Your apps → Web (</>)** → nickname `rheumatrack-web` → **Register app**. Copy the config values shown (apiKey, authDomain, projectId, appId, …).
5. In VS Code, copy `.env.example` to a new file `.env.local` (same folder) and fill in:

```
VITE_FIREBASE_API_KEY=AIza...
VITE_FIREBASE_AUTH_DOMAIN=rheumatrack-prod.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=rheumatrack-prod
VITE_FIREBASE_APP_ID=1:1234567890:web:abc123
VITE_FIREBASE_STORAGE_BUCKET=rheumatrack-prod.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=1234567890
```

> `.env.local` is in `.gitignore`; it will not be pushed to GitHub. These web values are not secrets (Security Rules protect the data), but keep them out of the repository anyway. Never put a service-account key in a `VITE_` variable.

6. Deploy the Security Rules and indexes (required — without them Firestore denies everything):

```
npx firebase login
npx firebase use --add          # pick your project, alias "prod"
npm run test:rules              # optional but recommended
npx firebase deploy --only firestore
```

7. Run the app against the real project and test sign-up, a check-in and an import:

```
npm run dev
```

How the app decides which backend to use (`src/data/firebaseConfig.ts`): emulator mode when `VITE_USE_EMULATORS=true`, otherwise the real project from the `VITE_FIREBASE_*` values, otherwise the “Setup required” screen.

```
  if (str('VITE_USE_EMULATORS') === 'true') {
    return {
      kind: 'emulator',
      config: {
        apiKey: 'demo-api-key',
        authDomain: `${EMULATOR_PROJECT_ID}.firebaseapp.com`,
        projectId: EMULATOR_PROJECT_ID,
        appId: 'demo-app-id',
      },
      authHost: str('VITE_EMULATOR_AUTH_URL') || 'http://127.0.0.1:9099',
      firestoreHost: str('VITE_EMULATOR_FIRESTORE_HOST') || '127.0.0.1',
      firestorePort: Number(str('VITE_EMULATOR_FIRESTORE_PORT') || 8080),
    };
  }
```

Optional later: App Check with reCAPTCHA Enterprise (`VITE_APPCHECK_RECAPTCHA_ENTERPRISE_KEY`), backups (Blaze plan) — see `docs/FIREBASE_SETUP.md`.

12. Push to GitHub from VS Code

1. Install Git (`winget install Git.Git`), reopen VS Code, and set your identity once:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

2. Click the **Source Control** icon (left bar, branch symbol) → **Initialize Repository**.
3. Check the file list: `node_modules`, `dist`, `.env.local` and `.emulator-data` must **not** appear (they are ignored by `.gitignore`). If `.env.local` appears, stop and add it to `.gitignore`.
4. Type a message such as `Initial commit: RheumaTrack` → click **Commit** (answer **Yes** to “stage all changes”).
5. Click **Publish Branch** → sign in to GitHub in the browser when asked → choose **Publish to GitHub private repository** (recommended) → name `rheumatrack`.
6. Open the repository link VS Code shows to confirm the files are on GitHub.
7. Later changes: Source Control → message → **Commit** → **Sync Changes** (push).

Terminal equivalent:

```
git init
git add .
git commit -m "Initial commit: RheumaTrack"
git branch -M main
git remote add origin https://github.com/<your-username>/rheumatrack.git
git push -u origin main
```

13. Deploy on Vercel

> Vercel hosts the website; Firebase still provides login and the database. The repository already contains `vercel.json` (Vite build, output `dist`, single-page-app rewrite so links like `/history` work on reload, and security headers).

1. Complete section 11 (Firebase project + rules deployed) and section 12 (code on GitHub).
2. Go to **https://vercel.com → Sign up with GitHub → Add New… → Project** → find `rheumatrack` → **Import**.
3. Framework preset: **Vite**. Build command `npm run build`, output directory `dist` (pre-filled from `vercel.json`).
4. Open **Environment Variables** and add each line of your `.env.local` (name and value), for Production and Preview: `VITE_FIREBASE_API_KEY`, `VITE_FIREBASE_AUTH_DOMAIN`, `VITE_FIREBASE_PROJECT_ID`, `VITE_FIREBASE_APP_ID` (+ the two optional ones). Do **not** add `VITE_USE_EMULATORS`.
5. Click **Deploy**. When it finishes, note the URL (e.g. `rheumatrack-xyz.vercel.app`).
6. Firebase console → **Authentication → Settings → Authorized domains → Add domain** → paste the Vercel domain (without https://). Needed for password-reset and verification links.
7. In Vercel → Project → **Settings → General → Node.js Version**: 22.x or 24.x (redeploy if you change it).
8. Open the live URL and repeat the manual checklist (section 10.2) for sign-up, check-in, import, sharing, and reload on `/history`.

Every **Sync Changes** (git push) to `main` redeploys automatically. Security-rule changes are deployed separately with `npx firebase deploy --only firestore`.

Alternative: Firebase Hosting with `npm run deploy` (uses `firebase.json`).

14. Troubleshooting

Symptom	Fix
`node` / `npm` not recognised	Reopen VS Code or refresh PATH (6.1).
“running scripts is disabled”	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or use `npm.cmd`.
“firebase-tools no longer supports Java version before 21”	Install Java 21 and make it first on PATH (6.1).
“Could not spawn java”	Java missing from PATH.
Port 8080 / 9099 / 4000 / 5173 in use	Close the other process or the old terminal; or change ports in `firebase.json`.
Sign-in shows “Network error”	Emulators not running — start `npm run dev:local`.
“Setup required” page	No `.env.local` and not in emulator mode.
“You do not have permission”	Rules not deployed to the real project, or a clinician without an accepted share.

Physician sees no invitations	Email not verified, role not granted, or did not sign out/in after the grant.
3D viewer blank	Enable hardware acceleration in the browser (WebGL).
Vercel page 404 on reload	`vercel.json` missing from the repo root.
Vercel shows “Setup required”	Environment variables missing — add them and redeploy.
`npm audit` warnings	Known; do not run `npm audit fix --force`.

15. Code walkthrough

15.1 Folder structure

```
rheumatrack/
├─ src/
│  ├─ main.tsx                entry: picks emulator / real Firebase / setup screen
│  ├─ app/                    App.tsx (routes), layout (sidebar, top bar), providers, route
guards
│  ├─ components/             UI primitives, charts, joint picker, filters
│  ├─ data/                   services.ts (interfaces), firestore/ (Firebase), memory/ (tests
only)
│  ├─ design/                 tokens.css (light + dark), base, components, layout CSS
│  ├─ domain/                 pure logic: joints, validation, metrics, repetitions,
correlation,
│  │                          symptoms, filters, export, reminders, calibration, synthetic,
analytics
│  ├─ features/               pages: auth, dashboard, joints, viewer3d, movement, symptoms,
history,
│  │                          analytics, reports, sharing, clinician, calibration, settings,
setup
│  ├─ hardware/ImuAdapter.ts  future sensor interface (not implemented)
│  └─ test-harness/           in-memory build for browser tests only
├─ firebase/                  firestore.rules, firestore.indexes.json
├─ matlab/                    MATLAB scripts + samples/
├─ reference/                 python/rt_analysis.py + golden datasets
├─ scripts/                   seed-emulator, grant-clinician, sample/golden generators
├─ tests/                     rules/ (Security Rules), e2e/ (Playwright)
├─ docs/                      architecture, data contract, security, setup, verification …
├─ firebase.json  .firebaserc  vercel.json  .env.example  package.json  vite.config.ts
tsconfig*.json
```

15.2 What happens when you import a file

1. `ImportPage.tsx` reads the file text in the browser.
2. `validateMovementFile()` (`domain/validation.ts`) parses CSV (PapaParse) or JSON and checks every rule, collecting row-level errors. Nothing is repaired:

```
  // Ordering and duplicates — reported, never repaired.
  for (let i = 1; i < samples.length; i++) {
    const prev = samples[i - 1] as AngleSample & { row: number };
    const cur = samples[i] as AngleSample & { row: number };
    if (cur.t === prev.t) {
      c.error(cur.row, 'timestamp', `Duplicate timestamp (same as row ${prev.row}).`);
    } else if (cur.t < prev.t) {
      c.error(cur.row, 'timestamp', `Timestamp is earlier than row ${prev.row}; samples must be
in ascending time order.`);
    }
  }
  if (c.hasErrors) return c.fail();
```

3. `computeMetrics()` (`domain/metrics.ts`) calculates every value from the validated samples. Note ROM = max − min:

```
export function computeMetrics(samples: readonly AngleSample[]): MovementMetrics {
  if (samples.length < 2) throw new Error('computeMetrics() requires at least two samples');
  const angles = samples.map((s) => s.angleDeg);
  let minDeg = Infinity;
  let maxDeg = -Infinity;
  for (const a of angles) {
    if (a < minDeg) minDeg = a;
    if (a > maxDeg) maxDeg = a;
  }
  const intervals = intervalsOf(samples);
  const interval = intervalStats(intervals);
  const reps = countRepetitions(angles);
  const first = samples[0] as AngleSample;
  const last = samples[samples.length - 1] as AngleSample;
  return {
    sampleCount: samples.length,
    minDeg,
    maxDeg,
    romDeg: maxDeg - minDeg,
    meanDeg: mean(angles),
    durationMs: last.t - first.t,
    interval,
    quality: qualityIndicators(intervals, interval.median),
    repetitions: reps.count,
    repetitionNote: reps.note,
  };
}
```

4. Repetitions use a hysteresis counter with thresholds at 30 % and 70 % of the observed range:

```
  const low = min + ANALYSIS_CONFIG.repLowFraction * rom;
  const high = min + ANALYSIS_CONFIG.repHighFraction * rom;

  type State = 'unknown' | 'below' | 'above';
  let state: State = 'unknown';
  let seenBelow = false;
  let count = 0;
  for (const a of angles) {
    if (a <= low) {
      if (state === 'above' && seenBelow) count++;
      state = 'below';
      seenBelow = true;
    } else if (a >= high) {
      state = 'above';
    }
  }
  return { count, note: null, lowThresholdDeg: low, highThresholdDeg: high };
}
```

5. On **Save**, the page calls `data.createSession()`. The Firestore implementation writes the samples in chunks plus the session document in one batch; the Security Rules check ownership, types, ranges and that `romDeg == maxDeg − minDeg`.
6. `RecordsProvider` listens to the user's sessions in real time, so the Dashboard, History, Joints and Trends pages update immediately.

15.3 The service interface (how pages talk to the backend)

```
export interface DataService {
  // Profile and role
  getProfile(uid: string): Promise<UserProfile | null>;
  createProfile(uid: string, input: ProfileInput): Promise<void>;
  updateProfile(uid: string, patch: Partial<Pick<UserProfile, 'displayName' | 'reminders' |
'preferredJoint' | 'preferredSide'>>): Promise<void>;
  subscribeProfile(uid: string, cb: (p: UserProfile | null) => void, onError: (e: Error) =>
void): Unsubscribe;
  /** Role as recorded server-side. Missing role document = patient. */
  getRole(uid: string): Promise<Role>;

  // Movement sessions (ownerUid = the patient whose records these are)
  subscribeSessions(ownerUid: string, cb: (s: MovementSession[]) => void, onError: (e: Error) =>
void): Unsubscribe;
  createSession(ownerUid: string, input: NewSessionInput, samples: readonly AngleSample[]):
Promise<string>;
  getSessionSamples(ownerUid: string, session: Pick<MovementSession, 'id' | 'chunkCount'>):
Promise<AngleSample[]>;
  updateSessionNotes(ownerUid: string, sessionId: string, notes: string): Promise<void>;
  deleteSession(ownerUid: string, session: Pick<MovementSession, 'id' | 'chunkCount'>):
Promise<void>;
```

15.4 Symptom ↔ ROM association

Pairs are made within ±2 h, then Spearman’s rank correlation is computed only with ≥ 10 pairs:

```
export function associate(pairs: readonly PairedObservation[], minPairs: number =
ASSOCIATION_CONFIG.minPairs): AssociationResult {
  const n = pairs.length;
  if (n < minPairs) {
    return { status: 'insufficient', n, reason: `At least ${minPairs} paired observations are
needed; ${n} available.` };
  }
  const x = pairs.map((p) => p.romDeg);
  const y = pairs.map((p) => p.symptom);
  const rho = spearman(x, y);
  if (rho === null) {
    return { status: 'insufficient', n, reason: 'One of the variables does not vary across the
paired observations, so no correlation can be computed.' };
  }
  return { status: 'ok', n, rho, pValue: spearmanPValue(rho, n), method: 'spearman' };
}
```

15.5 3D viewer

`features/viewer3d/`: `rig.ts` defines the skeleton proportions, joint pivots, camera framing and how an angle becomes a rotation; `geometry.ts` builds the smooth limb and torso shapes; `Figure.tsx` assembles the mannequin with nested pivot groups (so moving the shoulder carries the elbow and wrist), clickable joint balls and the observed-range arc; `Scene.tsx` sets up lights, floor, shadows and the smooth camera; `BodyViewer3D.tsx` adds the controls; `SamplePlayer3D.tsx` poses the figure from recorded samples.

```
export function poseRotation(joint: Joint, movement: Movement, side: Side, angleDeg: number):
Euler3 {
  const a = angleDeg * DEG;
  switch (joint) {
    case 'shoulder':
      if (movement === 'abduction') return [0, 0, sideSign(side) * a];
      return movement === 'extension' ? [a, 0, 0] : [-a, 0, 0];
    case 'hip':
    case 'wrist':
      return movement === 'extension' ? [a, 0, 0] : [-a, 0, 0];
    case 'elbow':
      return [-a, 0, 0];
    case 'knee':
      return [a, 0, 0];
    case 'ankle':
      return movement === 'plantarflexion' ? [a, 0, 0] : [-a, 0, 0];
  }
}
```

15.6 Security

All permissions are enforced in `firebase/firestore.rules`: per-user isolation, server-only roles, invite → accept → share, validation of every write, immutable metrics and calibration records. 18 tests in `tests/rules/firestore.rules.test.ts` try to break them (wrong user, changed IDs, self-granted roles, unverified emails, invalid values).

Appendix A — file index

File / folder	Purpose
`src/main.tsx`	Starts the app; chooses emulator, real project or setup screen.
`src/app/App.tsx`	All routes (patient, clinician read-only, auth) with lazy loading.
`src/app/providers/*`	Theme, Services, Auth (user, role, profile), Selection (joint/side/movement), Records (live data), Toasts.
`src/app/layout/*`	Sidebar, top bar, navigation, theme toggle, reminder banner.
`src/app/routes/guards.tsx`	ProtectedRoute, RoleRoute, GuestRoute.
`src/domain/joints.ts`	Joint/movement catalogue and labels.

`src/domain/validation.ts, timestamps.ts`	Import validation and strict ISO timestamp parsing.
`src/domain/metrics.ts, repetitions.ts`	ROM, mean, intervals, quality, repetition counting.
`src/domain/correlation.ts`	Pairing, Spearman, p-value.
`src/domain/analytics.ts, filters.ts`	Dashboard aggregates, history filters, date ranges.
`src/domain/export.ts`	CSV/JSON export formats.
`src/domain/symptoms.ts, reminders.ts, calibration.ts, synthetic.ts`	Symptom validation, reminder scheduling + .ics, static-hold checks, synthetic generator.
`src/data/services.ts`	Service interfaces used by all pages.
`src/data/firestore/*`	Firebase initialisation, auth service, Firestore data service.
`src/data/memory/memoryServices.ts`	In-memory implementation for tests.
`src/features/*`	One folder per page (see 15.1).
`firebase/firestore.rules`	Security Rules.
`scripts/seed-emulator.ts, grant-clinician.ts`	Demo data; clinician role management.
`matlab/*.m`	Generator, analyser, exporter, trend set, golden verification.
`reference/python/rt_analysis.py`	Independent reference implementation.
`tests/rules, tests/e2e, src/**/*.test.ts(x)`	Rules, browser and unit/integration tests.
`docs/*.md`	Detailed technical documentation.

Appendix B — command reference

```
npm install                          install dependencies
npm run dev:local                    emulators + app (local)
npm run seed                         demo accounts + synthetic data (local)
npm run dev                          app against the real project (.env.local)
npm run grant-clinician -- --emulator <email>        clinician role (local)
npm run grant-clinician -- --project <id> <email>    clinician role (real; needs key)
npm run typecheck | npm test | npm run build
npm run test:rules | npm run test:e2e:harness | npm run test:e2e
npx firebase deploy --only firestore                 deploy rules + indexes
npm run deploy                                       build + Firebase Hosting
```